

CampSite Coding Conventions

Team 5, CIS454. Jira story: TM05-41 (Coding Standard).

1. Purpose and enforcement

These conventions keep four people's code readable as one codebase. Every rule here either describes what the code on `main` already does, or is marked **new: adopt going forward** where the current code does not yet comply. Existing violations are listed so they can be fixed in passing, not as a blocker.

Most rules are enforced by tools, not by review:

Tool	Config	What it enforces
ruff (lint)	<code>pyproject.toml [tool.ruff]</code>	PEP 8 errors (E), pyflakes (F), import order (I), modern syntax for Python 3.12 (UP), bug-prone patterns (B). Line length 100.
ruff (format)	same	One formatting style; <code>ruff format --check</code> fails on any diff.
pytest	<code>pyproject.toml [tool.pytest.ini_options]</code>	Tests in <code>/tests</code> , <code>--strict-markers</code> , coverage report.
oxlint	<code>frontend/.oxlintrc.json</code>	React, TypeScript and oxc rules; <code>rules-of-hooks</code> is an error. We use oxlint, not ESLint.
tsc	<code>frontend/tsconfig.app.json</code>	<code>noUnusedLocals</code> , <code>noUnusedParameters</code> , <code>verbatimModuleSyntax</code> , <code>erasableSyntaxOnly</code> , <code>noFallthroughCasesInSwitch</code> .
GitHub Actions	<code>.github/workflows/ci.yml</code>	Runs all of the above on every push and PR.

The gate to run before pushing (the same checks CI runs):

```
ruff check . && ruff format --check . && pytest
cd frontend && npm run lint && npm run build
```

`npm run build` runs `tsc -b` before Vite, so it is the type check as well as the build.

2. Naming

Good names do most of the work of comments. The rules below follow the variable-naming guidance in the course text (Section III, Chapter 11), restated for our code.

Name the thing in the problem domain, not the implementation. A name should say what the value *means* to a camper or a map, not how it is stored. `public_access`, `site_type`, `last_run` and `geodesic_length_m` say what they are; names like `data`, `result`, `row` or `the_dict` only say what shape they are. Use them only for a value that truly has no meaning beyond its shape, in a few-line scope.

Length proportional to scope. A loop index over three lines can be `i`; a module constant read across the app needs a full name such as `MAX_RADIUS_MILES` or `METRIC_SRID`. If a name has to be explained in a comment at every use, it is too short.

Put computed-value qualifiers in one consistent place. Our constants put the qualifier first: `MAX_LIMIT`, `DEFAULT_LIMIT`, `MAX_SIMPLIFY`, `MAX_RADIUS_MILES`. Keep that order (`MAX_`, `MIN_`, `DEFAULT_` as a prefix) so related names sort together and read the same way. *Worth fixing:* `LIMIT_BEYOND` in `backend/api/layers.py` breaks the pattern; something like `MIN_LIMIT` or `LIMIT_ABOVE_MAX_AREA` would match.

Booleans read as true/false. Prefer `is_` / `has_` (Python) and `is` / `has` (TypeScript): `isSaved` in `SaveButton.tsx` is the model. An adjective that can only be true or false, like the `reservable` and `perennial` model fields, is also fine. A bare verb is not: `Layer.exact` and `Layer.clip` in `backend/api/layers.py` could be read as actions. **New: adopt going forward;** rename candidates are `is_exact` and `clips_to_viewport`.

No ambiguous abbreviations. Spell words out unless the short form is the standard name in our domain: `aoi`, `bbox`, `srid`, `geom`, `osm`, `ridb`, `nhd` and `padus` are fine; `cnt`, `tmp`, `res` and `val` are not. A scan of `main` found none of the latter.

Units in the names of physical quantities. Any variable, field or constant holding a distance, area, elevation, angle or duration carries its unit as a suffix: `length_m`, `radius_mi`, `elevation_m`, `max_tile_degrees`, `EARTH_RADIUS_M`. Use `_m`, `_km`, `_mi`, `_deg`, `_sq_deg`, `_s` consistently.

This rule exists because of a real bug class in this project. Our geometry is stored in EPSG:4326, so the raw `.length` of a geometry and `ST_Distance` on `geom` return *degrees of arc*, not metres, and inconsistently so: a degree of longitude is about 79 km at Adirondack latitude against about 111 km for latitude (`docs/architecture.md`, `docs/pipeline.md`). A bare `length` or `distance` gives no hint which unit you are holding. That is why trail length is stored as `length_m`, computed by `pipeline.geometry.geodesic_length_m`, and why TM05-42 is adding a metric projection for distance queries. A unit in the name makes the wrong value look wrong at the call site.

Worth fixing (**new: adopt going forward** for internal names):

- `backend/pipeline/adapters/ridb.py`, `fetch()`: local `radius` is in miles; call it `radius_mi`. (The API query key stays `"radius"` because RIDB defines it.)
- `backend/api/layers.py`: `MAX_SIMPLIFY` and `limit_for_bbox`'s local `area` are in degrees and square degrees; `MAX_SIMPLIFY_DEG` and `area_sq_deg` would say so. Public API parameter names (`simplify`, `bbox`) stay as documented in `docs/api.md`.

3. Python and Django

- **Style:** PEP 8 as enforced by ruff; formatting is whatever `ruff format` produces. Don't hand-format or add `# noqa` to dodge a rule without a comment saying why.
- **Imports:** sorted by ruff's isort. Our apps (`accounts`, `api`, `config`, `devdata`, `geodata`, `health`, `pipeline`) are first-party; add a new app to `known-first-party` in `pyproject.toml`.
- **Apps:** short, lowercase, one word where possible (`geodata`, `pipeline`, `accounts`).
- **Models:** singular PascalCase nouns (`Campsite`, `Trail`, `WaterFeature`, `PublicLand`, `IngestRun`). Choice sets are nested `TextChoices` classes with UPPER members and lowercase stored values (`Source.RIDB = "ridb"`). Fields are snake_case; timestamps are `created_at` / `updated_at`. Every ingested model inherits `SourceRecord`.
- **Management commands:** one verb or verb_noun per file (`ingest`, `seed`, `reset_db`, `build_sample`). Report failures by raising `CommandError`, and write output with `self.stdout`, not `print`.
- **Adapters:** a module per source (`ridb.py`, `osm_trails.py`, `nhd.py`). Classes are `<Source>` `<Dataset>Adapter` (`OsmTrailsAdapter`, `NhdFlowlinesAdapter`). The registry name, which is also the CLI argument, is `source-dataset` when a source has more than one dataset (`osm-trails`, `osm-campsites`, `nhd-flowlines`, `nhd-waterbodies`) and the bare source when it has one (`ridb`, `padus`). Adapters implement the `fetch` / `normalize` / `load` contract in `pipeline/adapters/base.py`; shared concerns (retries, reprojection, upserts, provenance) stay in the framework.
- **Type hints:** on public functions and method signatures, using built-in generics (`list[str]`, `tuple[float, ...]`) as ruff's UP rules require.

4. TypeScript and React

- **Components:** PascalCase, one per file, file named after it (`LayerPanel.tsx`, `SaveButton.tsx`, `pages/Discover.tsx`). Pages live in `src/pages/`, reusable pieces in `src/components/`.
- **Modules that aren't components:** lowercase / camelCase file names (`api.ts`, `layers.ts`, `popups.ts`, `regions.ts`).
- **Variables and functions:** camelCase; types and interfaces PascalCase (`SessionValue`, `Props`).
- **Hooks:** start with `use` (`useSession`) so `eslint's rules-of-hooks` can check them.
- **Imports of types:** `import type { ... }`, which `verbatimModuleSyntax` requires.
- **Colours:** never write a literal colour in a component or CSS file. Define a custom property in `src/theme.css` and use `var(--token)` in CSS. MapLibre paint properties can't read `var()`, so read the token with `token()` / `mapColors()` in `src/theme.ts`. The fallback literals inside `theme.ts` are the one sanctioned exception. CI's frontend build exists partly because a paint property once referenced a token that was never added.

5. Geospatial and SQL

- **Storage SRID is 4326.** Every geometry column is EPSG:4326 with a spatial (GiST) index. Sources in other projections (NHD in 4269, PADUS in 5070) are reprojected in `normalize()`; the framework does this from each adapter's `source_srid`.
- **Never measure in degrees.** Don't call `.length`, `.area` or `ST_Distance` on a 4326 geom and treat the answer as a distance. Use `geodesic_length_m` for line length and the metric distance helpers

from TM05-42 (or a cast to `geography`) for distances. Results go in unit-suffixed names (Section 2).

- **Areas of interest come from config** (`pipeline/regions.py`), never a hard-coded bounding box. Bounding boxes are (`min_lon`, `min_lat`, `max_lon`, `max_lat`) internally and `west`, `south`, `east`, `north` on the wire.
- **Idempotency on** (`source`, `source_id`). Every ingested model has a unique constraint on that pair, and `load()` upserts on it. Re-running an adapter must update rows, never duplicate them. `source_id` is the source's own stable ID, never our database ID.
- **Provenance**: every ingest run writes an `IngestRun`, and each row points at it via `last_run`.
- **Raw SQL** is a last resort; prefer the ORM and GeoDjango functions. If you need it, parameterise it, never format values into the string.

6. Testing

- Tests live in `/tests`, files `test_<area>.py` (`test_adapter_ridb.py`, `test_api_map_data.py`).
- Test functions name the behaviour and the expected result as a sentence:
`test_weak_password_is_rejected`, `test_login_with_valid_credentials_returns_a_token`.
- Shared fixtures go in `tests/conftest.py` (`env`, `sample_bbox`, `clean_registry`); a fixture used by one file stays in that file.
- Tests run against a real PostGIS database, not mocks of it. Network calls are faked with `unittest.mock` or sample payloads in `tests/fixtures/`; a test that genuinely hits a live service is marked `@pytest.mark.network` and is deselected by default, so an upstream outage can't turn CI red. `@pytest.mark.data` (in `tests/data_checks/`) is for checks against real seeded data. `--strict-markers` means an unregistered marker is an error.
- **Every PR needs tests for**: new behaviour (one passing case and the main failure case), any bug fix (a test that fails without the fix), and, for adapters, `normalize()` on a sample payload plus `load()` being idempotent on rerun. Never weaken a test or lower coverage to make CI pass.

7. Git and Jira

Branch names, commit format and the Jira workflow are in [docs/CONTRIBUTING.md](#); the short version:

- Branch `TM05-<issue>-short-description`; every commit message starts with the real issue key (TM05-13 add OSM trail adapter ...).
- Open a PR to `main`; it merges only after CI is green.
- Never commit `.env` files, tokens or credentials. Only `.env.example` with placeholders is tracked.

Worth fixing: 18 commits on `main` start with the placeholder `TM05-XX` (for example `d72e2f2`, `fa03e7e`).

Jira can't link them, so that work is invisible on any story. **New: adopt going forward**: no placeholder keys; if the story doesn't exist yet, create it first.

8. Comments, docstrings, errors and logging

- **Comments explain why, not what**. The code already says what it does. Comment the reason behind a non-obvious choice, a measured number, or a trap, the way `geodata/models.py` explains why distances on `geom` are in degrees. Delete a comment rather than let it go stale. (*Worth fixing*:

the `SourceAdapter` docstring in `pipeline/adapters/base.py` gives `"osm"` as an example `name`; the real names are `osm-trails` and `osm-campsites`.)

- **Docstrings:** every module, public class and public function gets one. The first line is a one-sentence summary; follow it with constraints, units and caveats when they matter.
- **Errors:** raise specific exception classes that subclass a per-area base (`AdapterError`, `RidbError`, `ArcGisError`), and chain with `raise ... from exc`. Never swallow an exception silently. Exception class names end in `Error`; this is **new: adopt going forward**, since `GeometryTypeMismatch`, `UnknownFeatureCode`, `UnknownPublicAccessCode` and `RidbUnavailable` don't yet.
- **Logging:** one module-level `logger = logging.getLogger(__name__)`; no `print` in application code. Never log secrets. The RIDB API key, for example, must never appear in logs or in `IngestRun.parameters`.

9. Team agreement

Each team member fills in their own row to record agreement with these conventions.

Team Member	Agreed (Y/N)	Date
Davis Stapleton		
Bleron Balidemaj		
Abdulahman Shaalan		
Sahaj Soni		